

Référence des Commandes Git

TP Gestion de versions

ISTA Mirleft

Introduction

Ce document présente l'ensemble des commandes Git nécessaires pour réaliser le TP de gestion de versions. Les commandes sont organisées par catégorie pour faciliter la consultation.

1 Configuration et Initialisation

Commande	Description et Utilité
<code>git config -global user.name "Nom"</code>	Configure le nom d'utilisateur global pour tous vos dépôts Git. Ce nom apparaîtra dans l'historique des commits.
<code>git config -global user.email "email"</code>	Configure l'adresse email globale associée à vos commits. Essentiel pour l'identification des contributions.
<code>git config -global core.editor vim</code>	Définit l'éditeur de texte par défaut pour Git (vim, nano, code, etc.).
<code>git config -list</code>	Affiche la liste complète de tous les paramètres de configuration Git (globaux et locaux).
<code>git config user.name</code>	Affiche la valeur d'un paramètre de configuration spécifique (ici le nom d'utilisateur).
<code>git init</code>	Initialise un nouveau dépôt Git dans le répertoire courant. Crée le dossier <code>.git</code> contenant toute la structure du dépôt.

2 Gestion des Fichiers et Commits

Commande	Description et Utilité
<code>git status</code>	Affiche l'état actuel du dépôt : fichiers modifiés, indexés, non suivis. Commande la plus utilisée pour connaître l'état de votre travail.
<code>git add <fichier></code>	Ajoute un fichier spécifique à la zone d'index (staging area) en préparation du prochain commit.
<code>git add .</code>	Ajoute tous les fichiers modifiés et nouveaux du répertoire courant à l'index.
<code>git commit -m "message"</code>	Crée un nouveau commit avec les fichiers indexés et le message de description fourni. Enregistre un instantané du projet.
<code>git commit -amend</code>	Modifie le dernier commit (message ou contenu). Utile pour corriger une erreur dans le dernier commit.
<code>git log</code>	Affiche l'historique complet des commits avec tous les détails (auteur, date, message, SHA-1).

<code>git log -oneline</code>	Affiche l'historique des commits de manière condensée (un commit par ligne avec SHA-1 court et message).
<code>git log -graph</code>	Affiche l'historique sous forme de graphe ASCII montrant les branches et les fusions.
<code>git log -graph -oneline -all</code>	Combine plusieurs options pour afficher un graphe complet, condensé, de toutes les branches.
<code>git log -n 5</code>	Limite l'affichage aux n derniers commits (ici 5).
<code>git log -stat</code>	Affiche l'historique avec des statistiques sur les fichiers modifiés (nombre de lignes ajoutées/supprimées).

3 Visualisation des Différences

Commande	Description et Utilité
<code>git diff</code>	Affiche les différences entre le répertoire de travail et l'index. Montre les modifications non indexées.
<code>git diff -staged</code>	Affiche les différences entre l'index et le dernier commit. Montre ce qui sera inclus dans le prochain commit.
<code>git diff <commit></code>	Affiche les différences entre le répertoire de travail et un commit spécifique.
<code>git diff <branche1> <branche2></code>	Compare deux branches et affiche leurs différences.
<code>git diff origin/main</code>	Compare la branche locale avec la branche distante correspondante.
<code>git show <commit></code>	Affiche les détails complets d'un commit spécifique (modifications, métadonnées).
<code>git show <tag></code>	Affiche les informations détaillées d'un tag annoté.

4 Annulation et Récupération

Commande	Description et Utilité
<code>git checkout - <fichier></code>	Annule les modifications d'un fichier dans le répertoire de travail et restaure la version du dernier commit.
<code>git restore <fichier></code>	Version moderne de la commande précédente. Restaure un fichier à son état du dernier commit.
<code>git reset HEAD <fichier></code>	Retire un fichier de la zone d'index (désindexe) sans perdre les modifications dans le répertoire de travail.
<code>git restore -staged <fichier></code>	Version moderne pour désindexer un fichier.
<code>git reset -soft HEAD~1</code>	Annule le dernier commit mais conserve les modifications dans l'index.
<code>git reset -mixed HEAD~1</code>	Annule le dernier commit et désindexe les modifications (par défaut).
<code>git reset -hard HEAD~1</code>	Annule complètement le dernier commit et supprime toutes les modifications. ATTENTION : destructif!

5 Gestion des Branches

Commande	Description et Utilité
----------	------------------------

<code>git branch</code>	Liste toutes les branches locales. La branche courante est marquée d'un astérisque (*).
<code>git branch <nom-branche></code>	Crée une nouvelle branche avec le nom spécifié, mais ne bascule pas dessus.
<code>git branch -d <nom-branche></code>	Supprime une branche locale fusionnée. Option sécurisée qui empêche la suppression si non fusionnée.
<code>git branch -D <nom-branche></code>	Force la suppression d'une branche même si elle n'est pas fusionnée.
<code>git branch -r</code>	Liste toutes les branches distantes (remote).
<code>git branch -a</code>	Liste toutes les branches (locales et distantes).
<code>git checkout <nom-branche></code>	Bascule sur la branche spécifiée et met à jour le répertoire de travail.
<code>git checkout -b <nom-branche></code>	Crée une nouvelle branche et bascule dessus immédiatement (combine branch et checkout).
<code>git switch <nom-branche></code>	Version moderne pour basculer entre les branches.
<code>git switch -c <nom-branche></code>	Version moderne pour créer et basculer sur une nouvelle branche.

6 Fusion et Rebasage

Commande	Description et Utilité
<code>git merge <branche></code>	Fusionne la branche spécifiée dans la branche courante. Crée un commit de fusion si nécessaire.
<code>git merge -no-ff <branche></code>	Force la création d'un commit de fusion même si une avance rapide (fast-forward) est possible. Conserve l'historique de la branche.
<code>git merge -squash <branche></code>	Fusionne tous les commits de la branche en un seul commit. L'historique est aplati.
<code>git merge -abort</code>	Annule une fusion en cours en cas de conflit et revient à l'état avant la fusion.
<code>git rebase <branche></code>	Réapplique les commits de la branche courante sur le sommet de la branche spécifiée. Linéarise l'historique.
<code>git rebase -i <commit></code>	Lance un rebasage interactif permettant de réorganiser, fusionner ou modifier des commits.
<code>git rebase -abort</code>	Annule un rebasage en cours et revient à l'état initial.
<code>git rebase -continue</code>	Continue le rebasage après avoir résolu des conflits.

7 Étiquetage (Tags)

Commande	Description et Utilité
<code>git tag</code>	Liste tous les tags du dépôt.
<code>git tag <nom-tag></code>	Crée un tag léger (simple pointeur) sur le commit courant.
<code>git tag -a <nom-tag> -m "msg"</code>	Crée un tag annoté avec un message. Stocke métadonnées complètes (auteur, date, message). Recommandé pour les versions.
<code>git tag -a <nom-tag> <commit></code>	Crée un tag annoté sur un commit spécifique (étiquetage après coup).

<code>git show <nom-tag></code>	Affiche les informations détaillées d'un tag (commit associé, message, diff).
<code>git tag -d <nom-tag></code>	Supprime un tag local.

8 Dépôt Distant - Configuration

Commande	Description et Utilité
<code>git remote</code>	Liste les noms des dépôts distants configurés.
<code>git remote -v</code>	Liste les dépôts distants avec leurs URLs complètes (fetch et push).
<code>git remote add <nom> <url></code>	Ajoute un nouveau dépôt distant avec un nom spécifique (généralement origin).
<code>git remote remove <nom></code>	Supprime un dépôt distant de la configuration.
<code>git remote rename <ancien> <nouveau></code>	Renomme un dépôt distant.
<code>git remote show <nom></code>	Affiche des informations détaillées sur un dépôt distant (branches de suivi, configuration).

9 Dépôt Distant - Synchronisation

Commande	Description et Utilité
<code>git clone <url></code>	Clone (copie complète) un dépôt distant vers votre machine locale. Crée un dossier avec le nom du dépôt.
<code>git clone <url> <dossier></code>	Clone un dépôt dans un dossier spécifique.
<code>git fetch</code>	Récupère toutes les modifications du dépôt distant et met à jour les branches distantes locales, SANS fusionner.
<code>git fetch <remote></code>	Récupère les modifications d'un dépôt distant spécifique.
<code>git fetch origin</code>	Récupère les modifications du dépôt origin.
<code>git pull</code>	Équivalent à <code>git fetch</code> suivi de <code>git merge</code> . Récupère et fusionne les modifications distantes dans la branche courante.
<code>git pull -rebase</code>	Récupère les modifications distantes et effectue un rebase au lieu d'un merge. Garde un historique linéaire.
<code>git push</code>	Publie les commits locaux vers le dépôt distant (sur la branche de suivi).
<code>git push <remote> <branche></code>	Publie une branche spécifique vers un dépôt distant.
<code>git push -u origin <branche></code>	Publie une branche et configure le suivi (upstream) pour les futurs push/pull. À utiliser lors de la première publication.
<code>git push -tags</code>	Publie tous les tags locaux vers le dépôt distant. Par défaut, <code>git push</code> ne publie pas les tags.
<code>git push origin <tag></code>	Publie un tag spécifique vers le dépôt distant.
<code>git push origin -delete <branche></code>	Supprime une branche du dépôt distant.
<code>git push -f</code>	Force la publication (push force). DANGEREUX : écrase l'historique distant. À utiliser avec précaution.

10 Fichiers Spéciaux

Fichier	Description et Utilité
<code>.gitignore</code>	Fichier texte listant les patterns de fichiers/dossiers à ignorer par Git. Un pattern par ligne. Exemples : <code>*.log, *.ini, __pycache__/</code>
<code>.git/</code>	Répertoire caché contenant toute la base de données Git : historique, branches, configuration locale, index, etc.
<code>.git/config</code>	Fichier de configuration locale du dépôt (prioritaire sur la config globale).
<code>~/.gitconfig</code>	Fichier de configuration globale de l'utilisateur (Linux/Mac).

11 Commandes Utiles Supplémentaires

Commande	Description et Utilité
<code>git stash</code>	Sauvegarde temporairement les modifications en cours dans une <code>réserve</code> pour nettoyer le répertoire de travail.
<code>git stash pop</code>	Récupère les modifications sauvegardées et les applique au répertoire de travail.
<code>git stash list</code>	Liste toutes les sauvegardes temporaires (stashes).
<code>git clean -fd</code>	Supprime tous les fichiers non suivis du répertoire de travail. <code>-f</code> = force, <code>-d</code> = répertoires.
<code>git rm <fichier></code>	Supprime un fichier du répertoire de travail ET de l'index.
<code>git rm -cached <fichier></code>	Retire un fichier de l'index mais le conserve dans le répertoire de travail.
<code>git mv <source> <destination></code>	Renomme ou déplace un fichier versionné.
<code>git archive -format=zip HEAD > projet.zip</code>	Crée une archive du projet à partir d'un commit (généralement un tag).
<code>git blame <fichier></code>	Affiche qui a modifié chaque ligne d'un fichier et quand.
<code>git help <commande></code>	Affiche l'aide détaillée d'une commande spécifique.
<code>git -version</code>	Affiche la version de Git installée.

12 Syntaxe des Références

Référence	Description
<code>HEAD</code>	Pointeur vers le commit courant (sommet de la branche actuelle).
<code>HEAD~1</code> ou <code>HEAD~</code>	Le premier parent du commit courant (commit précédent).
<code>HEAD~2</code>	Le grand-parent du commit courant (2 commits en arrière).
<code>HEAD~n</code>	Le n-ième ancêtre du commit courant.
<code>HEAD^</code>	Le premier parent du commit courant (identique à <code>HEAD~</code> pour les commits simples).

HEAD~2	Le deuxième parent d'un commit de fusion.
<branche>	Référence au dernier commit d'une branche.
origin/<branche>	Référence à une branche distante (tracking branch).
<SHA-1>	Référence à un commit spécifique par son hash (complet ou court).
<tag>	Référence à un commit étiqueté.

Notes Importantes

- **Commandes destructives** : Les commandes comme `git reset -hard`, `git push -f`, `git clean` peuvent supprimer définitivement des données. Utilisez-les avec précaution.
- **Branches de suivi** : Utilisez toujours l'option `-u` lors du premier push d'une branche pour configurer le suivi automatique.
- **Messages de commit** : Écrivez des messages clairs et descriptifs. Format recommandé : verbe à l'infinitif + description concise.
- **.gitignore** : Ajoutez ce fichier dès le début du projet pour éviter de versionner des fichiers inutiles.
- **Fetch vs Pull** : `git fetch` est plus sûr car il permet de vérifier les modifications avant de fusionner.
- **Rebase vs Merge** : Utilisez `merge` pour préserver l'historique complet, `rebase` pour un historique linéaire.

Ce document de référence accompagne le TP Git - Gestion de versions